

Deploying PublishAll

PublishAll is a standard Next.js app. It runs anywhere Node.js runs. Below are four common paths — pick whichever suits you.

Before any deployment, have these ready:

- A **PostgreSQL** database URL
 - Your **AI provider** credentials (`LLM_BASE_URL` , `LLM_API_KEY` , `LLM_MODEL`)
 - Two random secrets for `NEXTAUTH_SECRET` and `AUTH_SECRET` (`openssl rand -base64 32`)
-

Option 1 — Vercel (easiest)

1. Push this code to a Git repository (GitHub/GitLab/Bitbucket).
2. Go to vercel.com → **New Project** → import the repo.
3. Under **Environment Variables**, add everything from `.env.example` (plus `NEXTAUTH_URL` = your Vercel URL).
4. Deploy.
5. After the first deploy, run the database schema push once from your machine:

```
bash
DATABASE_URL="your-prod-url" yarn prisma db push
```

Vercel is serverless. Use a Postgres provider that supports pooled connections (Neon, Supabase) for best results.

Option 2 — Railway / Render (managed servers)

1. Create a new project and add a **PostgreSQL** plugin/instance — copy its connection string.
 2. Create a **web service** from this repo.
 3. Set the build command to `yarn install && yarn prisma generate && yarn build` and the start command to `yarn start`.
 4. Add all environment variables.
 5. After first deploy, run `yarn prisma db push` (Railway/Render both offer a one-off shell or you can run it locally against the prod URL).
-

Option 3 — Docker (self-host anywhere)

Create a `Dockerfile` in the project root:

```
FROM node:20-alpine
WORKDIR /app
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile
COPY . .
RUN yarn prisma generate && yarn build
EXPOSE 3000
CMD ["sh", "-c", "yarn prisma db push && yarn start"]
```

Build and run:

```
docker build -t publishall .
docker run -p 3000:3000 --env-file .env publishall
```

Option 4 — Any Node server (VPS, bare metal)

```
yarn install
yarn prisma generate
yarn prisma db push
yarn build
yarn start           # serves on port 3000
```

Put Nginx or Caddy in front of it for HTTPS and a domain.

Notes for portability

This project was originally created on the Abacus.AI platform. Two small, **optional** platform hooks are harmless off-platform but can be removed cleanly when self-hosting:

1. In `app/layout.tsx` — a `<script src="https://apps.abacus.ai/chatllm/appllm-lib.js" />` tag in `<head>`. Safe to delete.
2. `instrumentation-client.js` — a client error reporter that beacons to an Abacus path. Safe to delete or replace with your own (e.g. Sentry).

Nothing else references the original platform. The AI provider is fully configurable via the `LLM_*` environment variables (see `README.md`).